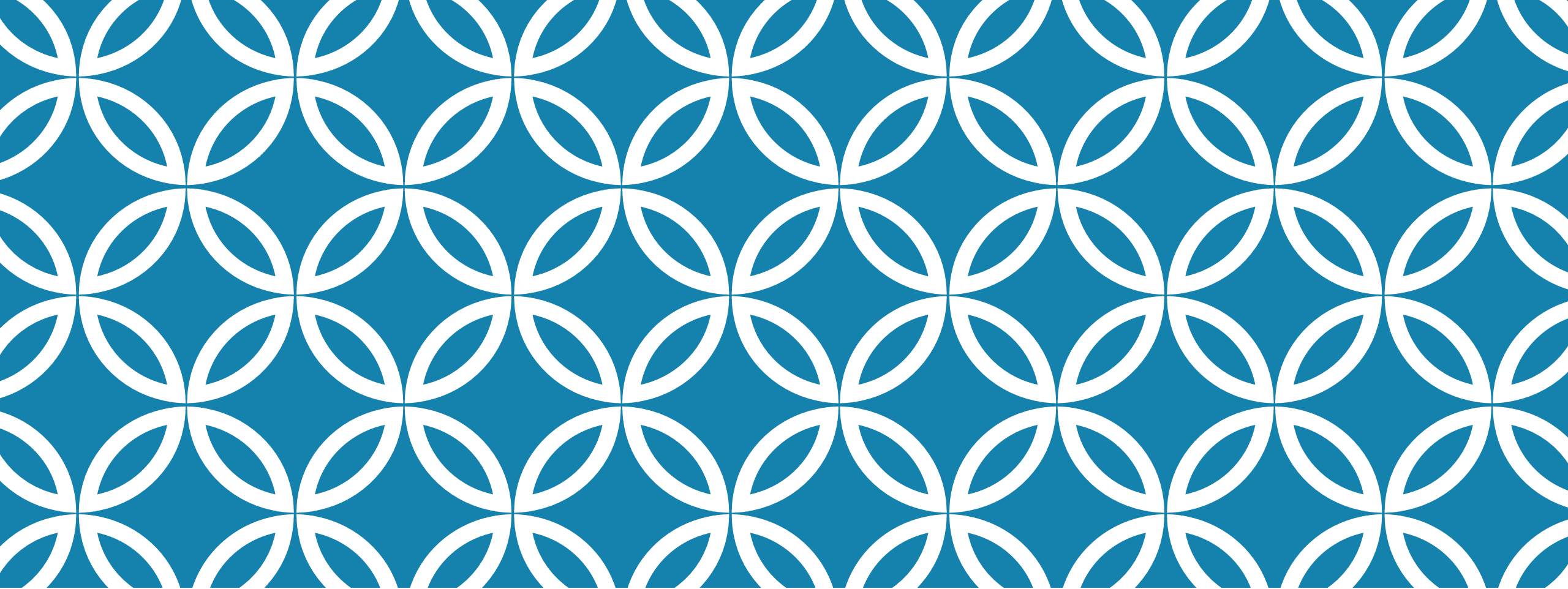




Programowanie asynchroniczne w C # - czy na pewno wiesz, jak go prawidłowo używać?

Beata Zalewa



18.11.2019

O MNIE



Konsultant, Microsoft Certified Trainer, administrator, programista, freelancer, czasami magik.



Od początku kariery związana z technologiami Microsoft.



Prywatne życie w liczbach: 1 mąż, 1 córka, 1 kot i 2 psy. Moje hobby to kryminały i fotografia.



Email: info@zalnet.pl



Skype: beata.zalewa



WWW: <https://zalnet.pl/>

<https://zalnet.pl/blog>

AGENDA

DLACZEGO?	Dlaczego potrzebujemy ich używać?
CO?	Co dokładnie robią async i await?
JAK?	Jaki i kiedy ich używać?
Q&A	

INTRO

- Kod jest wykonywany, aby zwrócić użytkownikowi rezultat.
- Zwykle jest to seria kroków, które na końcu prowadzą do ostatecznego wyniku.
- Wykonanie każdego z kroków zajmuje zazwyczaj trochę czasu, a ich synchroniczne uruchamianie jeden po drugim może zająć dużo więcej niż tylko trochę czasu.
- Co w sytuacji, gdy nie mamy tego czasu?

INTRO

- Aplikacje współbieżne odgrywają coraz ważniejszą rolę.
- Producenci dodają do komputerów i serwerów nowe rdzenie zamiast zwiększać moc obliczeniową pojedynczej jednostki.
- Słowa kluczowe *async* i *await* zostały wprowadzone w Visual Studio 2012 w celu uproszczenia zarządzania metodami asynchronicznymi.
- Jak najlepiej używać tych słów kluczowych w celu tworzenia wydajnego kodu wolnego od wielu nieuchwytnych błędów, które zamieszkują świat programowania asynchronicznego.



DLACZEGO?

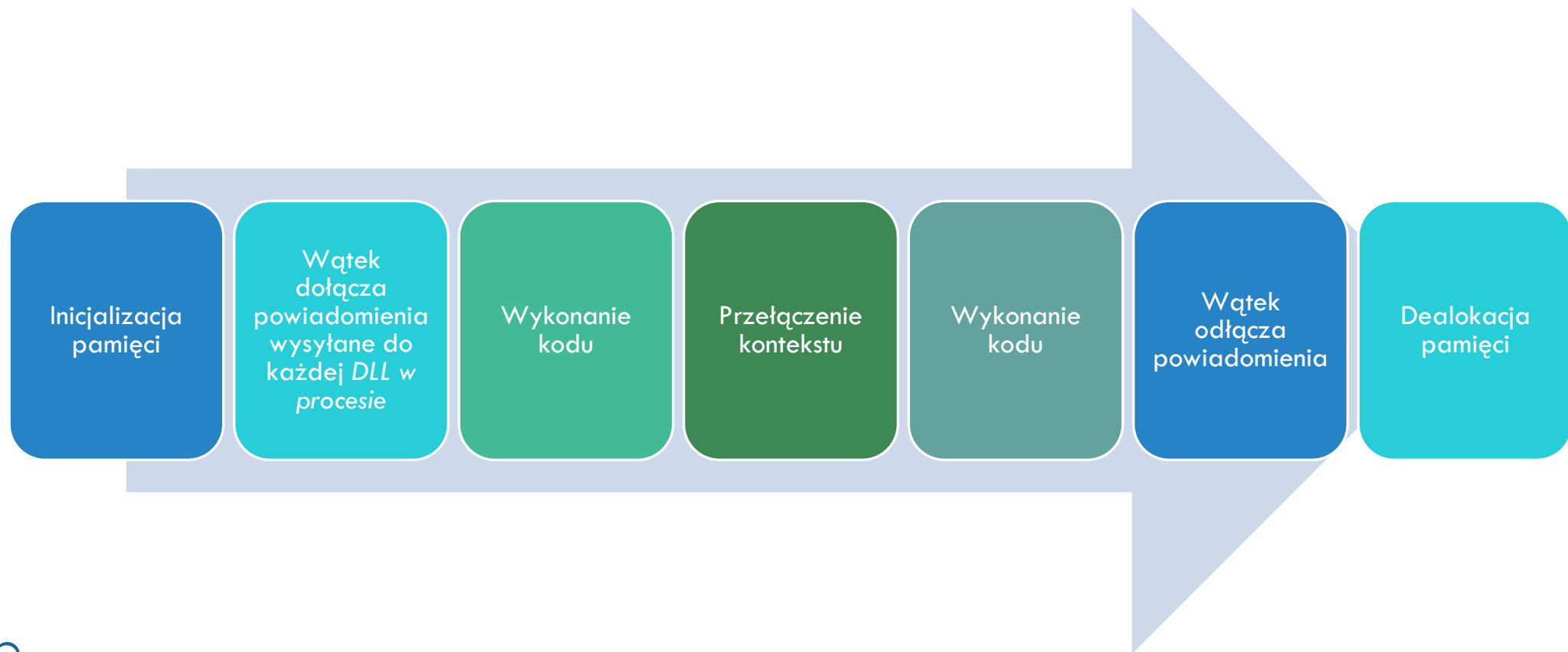
DLACZEGO? PODSTAWY

- OS został stworzony, aby uruchamiać *aplikacje*
- Te aplikacje są uruchamiane w *procesach*
- Procesy są dzielone na mniejsze jednostki *app domains*
- App domains uruchamiają *wątki*
- Proces w uruchomionym programie to np. .NET CLR host
- App domains dostarczają izolacji pomiędzy nimi i mogą być unikalnie konfigurowane, ładowane, wyładowywane i zabezpieczane
- Wątki umożliwiają zarządzanie wykonywaniem kodu

DLACZEGO? CO TO JEST *WĄTEK (THREAD)* ?

- W IT, wątek jest najmniejszą jednostką przetwarzania, która może być uruchamiana wg harmonogramu systemu operacyjnego
- Implementacja wątków i procesów różni się pomiędzy systemami operacyjnymi, ale w większości przypadków wątek jest zawarty wewnątrz *procesu*
- Wiele wątków może istnieć w tym samym procesie i współdzielić zasoby takie jak *pamięć*, podczas gdy inne procesy nie mogą współdzielić tych zasobów

DLACZEGO? ŻYCIE *JEDNEGO* WĄTKU



DLACZEGO? STAN *WĄTKU*

- Każdy wątek istnieje w określonym stanie wykonywania w danym czasie
- System operacyjny przechowuje stan wątku wewnątrz tzw. thread kernel object field "state"
- System operacyjny używa stanów, które są odpowiednia do wydajności
- Te stany to:

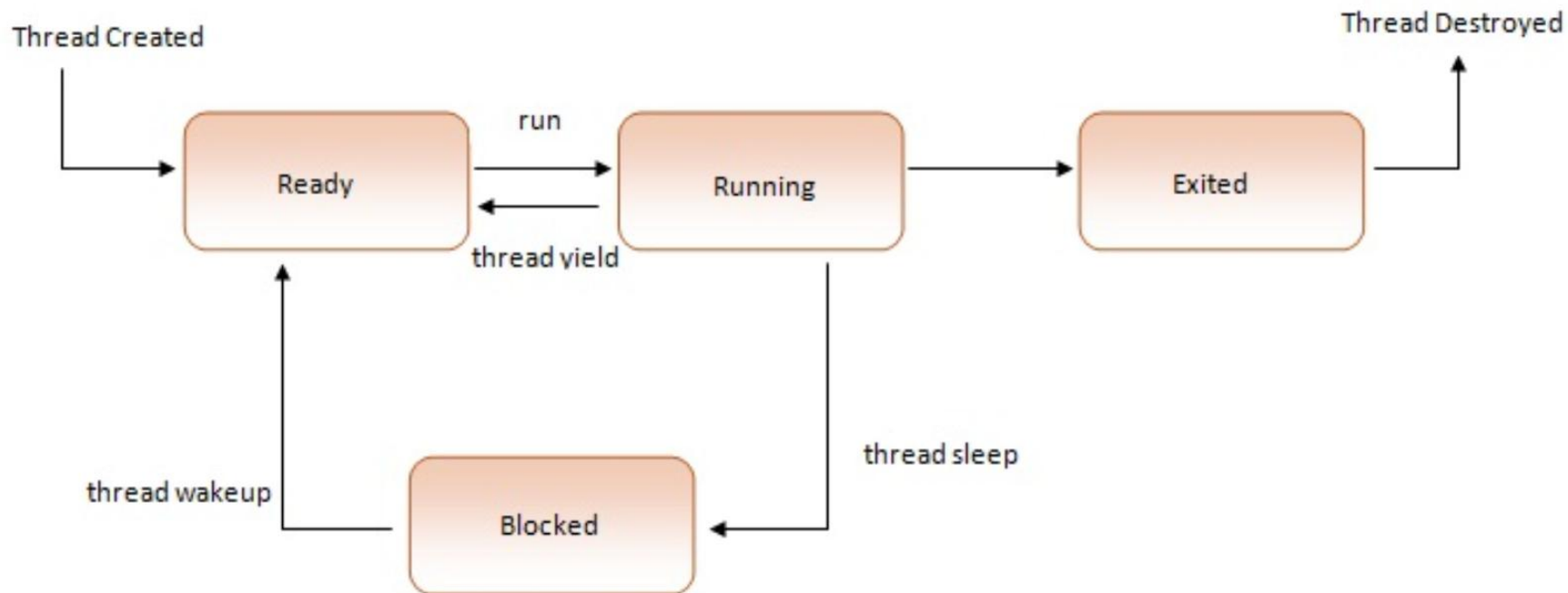
Running – wątek używa CPU

Blocked – wątek czeka na dane wejściowe

Ready – wątek jest gotowy do uruchomienia (ma status różny od Blocked lub Running)

Exited – wątek został zakończony, ale jeszcze nie został usunięty

DLACZEGO? STAN *WĄTKU*



Thread State Diagram

Źródło: www.codeproject.com



DEMO

Wątki

DLACZEGO? POTRZEBNE NAM SĄ *PULE WĄTKÓW (THREAD POOL)*

- Wszyscy wiemy, że wątki mają spory narzut
- Aby sobie z tym poradzić, CLR wprowadził *thread pool*
- *Thread pool* startuje jako pusty
- W miarę pojawiania się zadań, tworzone są nowe wątki
- Kiedy wątek skończy swoją pracę, jest zwracany do puli wątków i może zostać ponownie użyty
- Czasami trzeba pójść na kompromis:
 - Mniejszy narzut (mniej alokowanej pamięci)
 - Potrzeba mniej czasu na alokację wątku
 - Jakkolwiek, mniej wątków jest zaplanowanych w tym samym czasie



DEMO

Pule wątków

DLACZEGO? ZADANIA (TASKS)

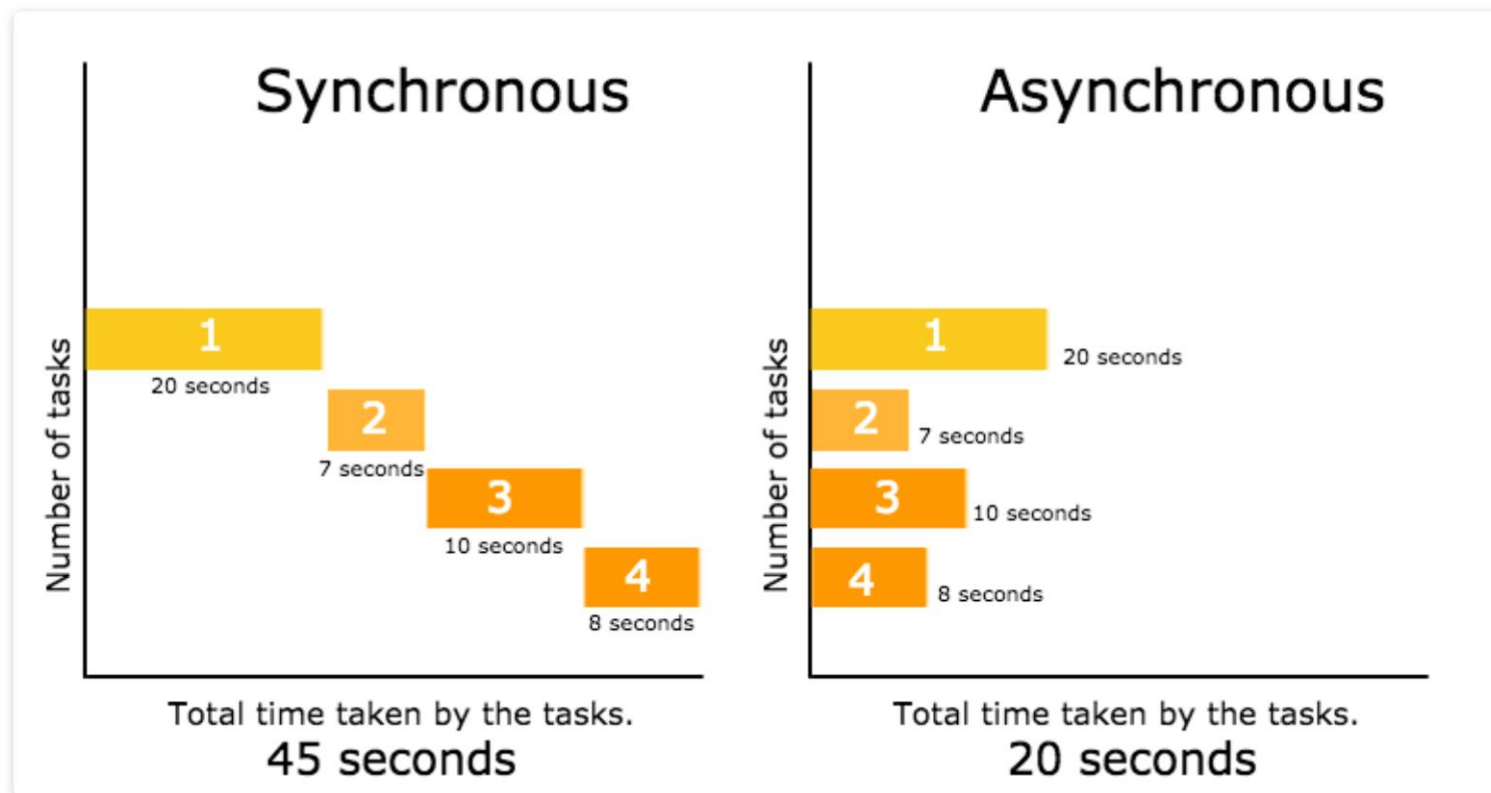
- Dzięki zadaniom łatwiej jest pracować z wątkami i pulami wątków
- Łatwo można je spauzować (WAIT)
- Automatyczna możliwość anulowania
- Prosty dostęp do rezultatu wykonania działania
- Powiązane zadania (jedno zadanie startuje kiedy inne się kończy)
- Zadanie zagnieżdżone
- Zadanie wykonywane współbieżnie



DEMO

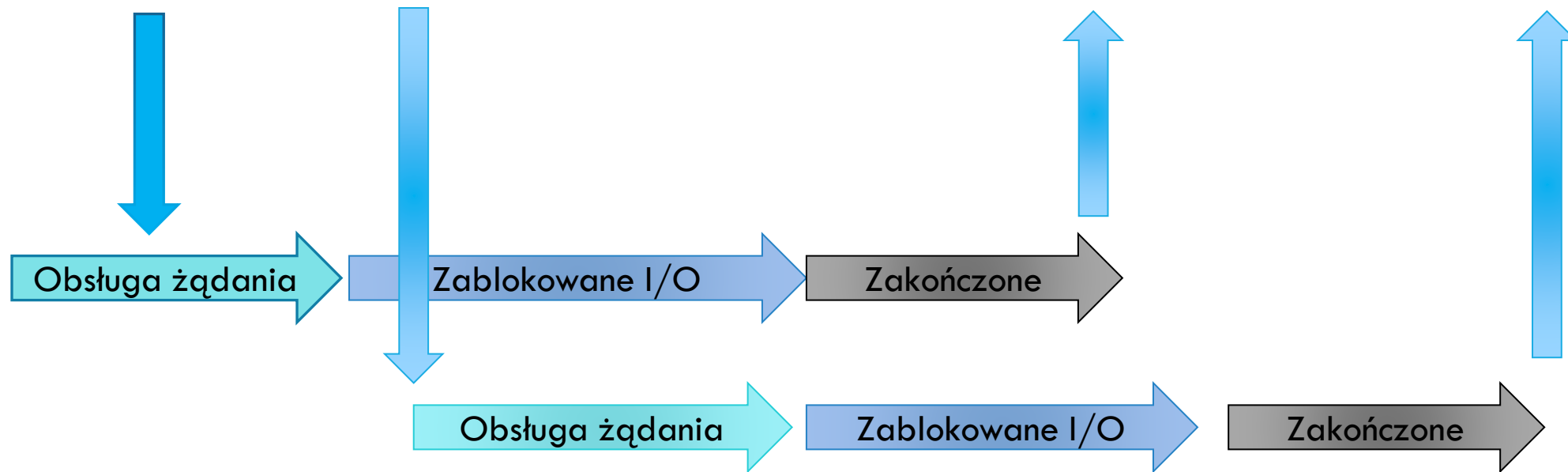
Zadania

DLACZEGO? SYNCHRONICZNIE VS ASYNCHRONICZNIE

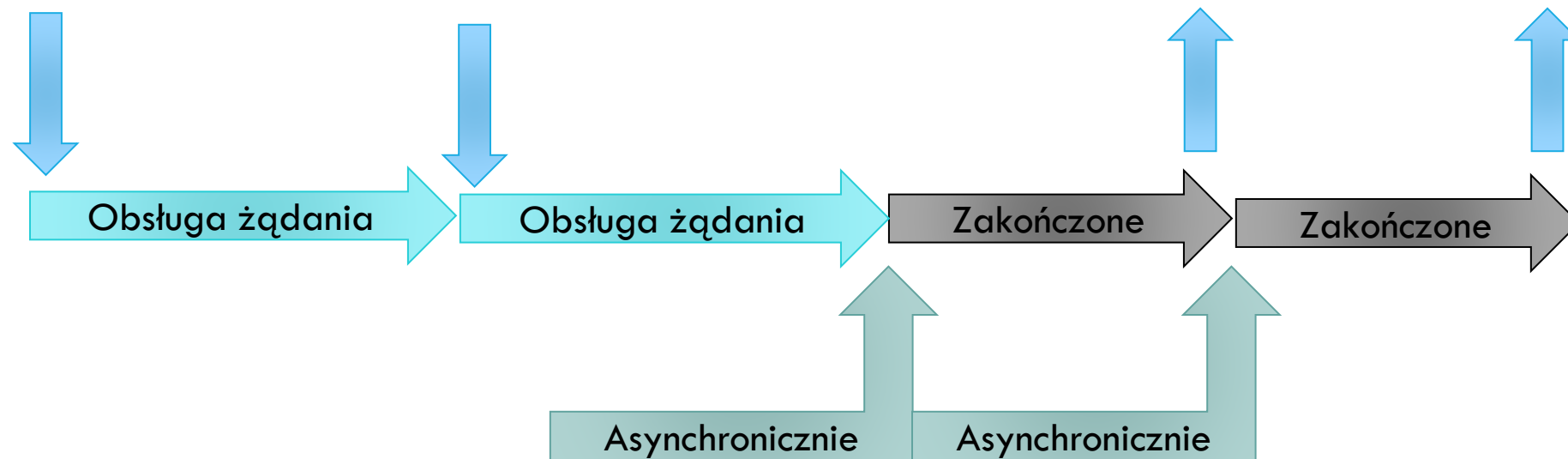


Źródło: www.phpmind.com

DLACZEGO? SYNCHRONICZNIE: DWA WĄTKI



DLACZEGO? ASYNCHRONICZNIE: JEDEN WĄTEK



A photograph of a workspace on a wooden desk. In the background, a silver laptop is open. In the foreground, a spiral-bound notebook with a grid pattern is open, and a black pen with the brand name 'PRECISE' and a logo is lying on it. The text 'DEMO' is overlaid on the notebook.

DEMO

Asynchroniczność



CO?

CO? ASYNC

- Oczekuje użycia *await*
- Nie tworzy nowego wątku
- Po *await* może lub nie użyć tego samego wątku (pula wątków jest w to zaangażowana, więc wątki są ponownie używalne)
- Jeśli `SynchronizationContext` istnieje, będzie zwracał do tego wątku
- Możemy także zmodyfikować to zachowanie używając `ConfigureAwaitAwait`
- Najprościej ... używaj “`yield`” dla wątków!

CO? ASYNC

- Oczekuje użycia *await*
- Nie tworzy nowego wątku
- Po *await* może lub nie użyć tego samego wątku (pula wątków jest w to zaangażowana, więc wątki są ponownie używalne)
- Jeśli `SynchronizationContext` istnieje, będzie zwracał do tego wątku
- Możemy także zmodyfikować to zachowanie używając `ConfigureAwaitAwait`
- Najprościej ... używaj “*yield*” dla wątków!



DEMO
async await



JAK?

JAK?

1 reference

```
private static async Task<int> ZadanieKtoreCzytaStrone()  
{  
    var klient = new HttpClient();  
    var strona = await klient.GetStringAsync(witryna);  
    return strona.Length;  
}
```

JAK?

- Pracuj z wieloma wątkami, nie sprzedawaj się tanio!
- Ograniczenia obliczeniowe nie korzystają tak bardzo z asynchronicznego wzorca, jak mogłoby się wydawać, z wyjątkiem zwalniania głównego kontekstu (zazwyczaj wątku interfejsu użytkownika)
- I / O mają ogromne zalety
- Async nie tworzy nowego wątku. Zamiast tego ustanawia *stan maszyny* (ang. Machine state) i sprawia, że wątek może być ponownie użyty i ponownie dołączony
- Await nie przypomina Wait(), ponieważ nie blokuje i pozwala na recykling wątków
- Jeśli masz asynchroniczne operacje I / O, użyj go! Async Task<> jest twoim przyjacielem.

A photograph of a workspace featuring a silver laptop, a spiral-bound notebook with a grid pattern, and a black pen resting on the notebook. The items are on a wooden desk. The text 'DEMO Asynchroniczny ThreadPool' is overlaid on the bottom right of the image.

DEMO
Asynchroniczny
ThreadPool

TIP

Use **Thread.Sleep** when you want to block the current thread.

Use **Task.Delay** when you want a logical delay without blocking the current thread.

Email: info@zalnet.pl

Demo: <https://github.com/bzalewa/4Developers2019>

Q & A

Resources:

<https://docs.microsoft.com>

Internet

Dziękuję😊